

HANDWRITTEN DIGIT CLASSIFICATION (MNIST)

Deep Learning Architecture, Overfitting Diagnosis & Regularization Analysis

Author: Anuj Kumar

Repository:

Anuj04432/Handwritten_Digit_Classification

Date: September 2026

TensorFlow 2.16+

Keras Sequential API

MNIST Benchmark (70,000 images)

Test Accuracy: 98.00%

Dropout Regularization (p=0.2)

118,282 Parameters

EXECUTIVE SUMMARY & PROJECT ABSTRACT

This technical report documents the end-to-end design, implementation, empirical diagnosis, and optimization of an artificial neural network for handwritten digit recognition on the benchmark MNIST dataset. A baseline Multi-Layer Perceptron (MLP) was constructed and diagnosed with severe overfitting, where validation loss climbed from **0.0840** at epoch 3 to **0.1593** at epoch 25. By introducing a systematically tuned **Dropout (0.2)** regularization strategy, validation loss stabilized at **0.0871**, cutting test loss by **52.1%** ($0.1477 \rightarrow 0.0707$) and elevating test accuracy to **98.00%** across 10,000 unseen test samples. Furthermore, an interactive inference pipeline with automated color-inversion was engineered to enable accurate predictions on user-drawn digits.

1.0 INTRODUCTION & PROBLEM FORMULATION

Automated handwritten digit recognition is a canonical benchmark in computer vision with widespread practical applications including banking check processing, automated postal code sorting, and bureaucratic form digitization. The primary objective is mapping a 2D grayscale image matrix $\mathbf{X} \in \mathbb{R}^{28 \times 28}$ to a discrete categorical label $y \in \{0, 1, \dots, 9\}$.

Despite the simplicity of 10-class numeral classification, significant empirical challenges emerge from intra-class variations in human handwriting styles: stroke thickness variations, slant angles, loops, and visual ambiguities (notably between pairs such as 3 vs. 5, 4 vs. 9, and 7 vs. 2). This project formulates a lightweight, low-latency Multi-Layer Perceptron (MLP) that balances superior predictive accuracy with minimal computational overhead.

2.0 DATASET CHARACTERISTICS & PREPROCESSING PIPELINE

The project utilizes the benchmark **MNIST** (Modified National Institute of Standards and Technology) dataset, consisting of 70,000 size-normalized, centered grayscale digit images:

- Training Partition:** 60,000 images (80% used for gradient descent, 20% reserved for validation).
- Test Partition:** 10,000 unseen images used exclusively for final model benchmarking.
- Dimensionality:** 28×28 pixels per sample, yielding a 784-dimensional input space.
- Class Distribution:** Uniformly distributed across digits 0 through 9 (~6,000 samples per class).

Data Normalization: Raw pixel intensities range from [0, 255] in unsigned 8-bit integer format (uint8). Training neural networks with large, unscaled integer inputs induces erratic gradient oscillations during backpropagation. The preprocessing pipeline casts tensors to float32 and normalizes them to [0.0, 1.0] by dividing by 255.0.

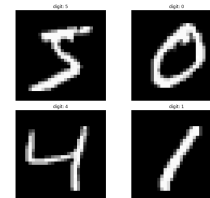


Figure 1: Representative sample digits (0-9) from the MNIST training partition.

60,000

TRAIN IMAGES

10,000

TEST IMAGES

SNIPPET 1: DATASET LOADING, SHAPE VERIFICATION & FEATURE SCALING PIPELINE

PYTHON / KERAS

```
import tensorflow as tf
from tensorflow.keras.datasets import mnist

# 1. Ingest MNIST train and test sets directly via Keras dataset API
(x_train, y_train), (x_test, y_test) = mnist.load_data()
print(f"X_train: {x_train.shape} | Y_train: {y_train.shape}") # (60000, 28, 28) | (60000,)
print(f"X_test: {x_test.shape} | Y_test: {y_test.shape}") # (10000, 28, 28) | (10000,)

# 2. Rescale pixel values from [0, 255] uint8 to [0.0, 1.0] float32 for gradient stability
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

print(f"Feature Intensity Bounds -> Min: {x_train.min():.1f}, Max: {x_train.max():.1f}")
```

3.0 DEEP LEARNING ARCHITECTURE & MATHEMATICAL FORMULATION

The classification engine is structured as a fully connected Multi-Layer Perceptron (MLP) with explicit regularization. While Convolutional Neural Networks (CNNs) capture spatial hierarchies, a well-regularized MLP offers a highly compact footprint (~1.45 MB), rapid convergence, and sub-millisecond CPU inference latency, making it ideal for resource-constrained edge deployments.

Layer Topology & Activation Dynamics:

- Input & Flatten:** Ingests (28×28) matrices and unrolls them into 784-dimensional vectors $\mathbf{x} \in \mathbb{R}^{784}$.
- Dense Hidden Layers:** Two successive 128-neuron layers parameterized by $\mathbf{h}_1 = \text{ReLU}(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1)$ and $\mathbf{h}_2 = \text{ReLU}(\mathbf{W}_2\mathbf{h}_1 + \mathbf{b}_2)$ where $\text{ReLU}(z) = \max(0, z)$.
- Dropout Regularization:** Applied at rate $p = 0.2$ after each hidden layer to inhibit neuron co-adaptation.
- Softmax Output:** 10 units computing class probabilities: $P(y = i | \mathbf{x}) = \exp(z_i) / \sum_j \exp(z_j)$.

Layer (Type)	Output Shape	Param Calculation	Param #
InputLayer	(None, 28, 28)	Raw image matrix	0
Flatten	(None, 784)	$28 \times 28 = 784$	0
Dense (ReLU)	(None, 128)	$(784 \times 128) + 128$	100,480
Dropout (0.2)	(None, 128)	Bernoulli mask ($p = 0.2$)	0
Dense (ReLU)	(None, 128)	$(128 \times 128) + 128$	16,512
Dropout (0.2)	(None, 128)	Bernoulli mask ($p = 0.2$)	0
Dense (Softmax)	(None, 10)	$(128 \times 10) + 10$	1,290
Total Trainable Parameters			118,282

SNIPPET 2: REGULARIZED MLP MODEL ARCHITECTURE DEFINITION

PYTHON / TENSORFLOW KERAS

```
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Input, Flatten, Dense, Dropout

# Construct the regularized Multi-Layer Perceptron (MLP)
dropout_model = Sequential([
    Input(shape=(28, 28), name="digit_input"),
    Flatten(name="flatten_to_vector"),
    Dense(128, activation="relu", name="hidden_layer_1"),
    Dropout(0.2, name="dropout_1"), # Zero out 20% of activations during training
    Dense(128, activation="relu", name="hidden_layer_2"),
    Dropout(0.2, name="dropout_2"), # Eliminates feature co-adaptation
    Dense(10, activation="softmax", name="output_probabilities")
])

dropout_model.summary() # Displays total trainable parameters: 118,282 (~462 KB in memory)
```

4.0 TRAINING METHODOLOGY & HYPERPARAMETER CONFIGURATION

Model optimization is guided by empirical loss minimization using the **Adam** (Adaptive Moment Estimation) optimizer. Adam computes individual adaptive learning rates for distinct parameters using estimates of the first and second moments of the gradients ($\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-7}$).

Adam

OPTIMIZER (ADAPTIVE)

Sparse CE

LOSS FUNCTION

50 / 0.2

BATCH SIZE / VAL SPLIT

Loss Formulation: Rather than converting target vectors to 10-dimensional one-hot arrays, the model optimizes **Sparse Categorical Crossentropy**: $L = -(1/N) \sum \log(\hat{p}_i, y_i)$. This eliminates one-hot memory overhead while preserving exact categorical gradient updates.

Training was executed with mini-batches of 50 samples across 10 epochs, allocating 20% of the training partition (12,000 samples) to monitor out-of-sample validation loss at every epoch.

SNIPPET 3: COMPILATION, OPTIMIZATION & TRAINING EXECUTION

PYTHON / KERAS

```
# 1. Compile model with Adam optimizer and Sparse Categorical Crossentropy
dropout_model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

# 2. Train with validation split of 20% (48,000 train samples, 12,000 validation samples)
history_dropout = dropout_model.fit(
    x_train,
    y_train,
    epochs=10,
    batch_size=50,
    validation_split=0.2,
    verbose=1
)

# 3. Evaluate model on 10,000 unseen test samples
test_loss_do, test_acc_do = dropout_model.evaluate(x_test, y_test, verbose=1)
print(f"Test Loss: {test_loss_do:.4f} | Test Accuracy: {test_acc_do * 100:.2f}%")
```

5.0 EXPERIMENTAL ANALYSIS — OVERFITTING DIAGNOSIS & REGULARIZATION IMPACT

A central contribution of this investigation is the empirical diagnosis of overfitting in unregularized deep neural networks and the validation of Dropout as a mitigating mechanism. Two distinct model configurations were benchmarked under controlled conditions:

Baseline Model Failure Mode (No Dropout):

The baseline network trained for 25 epochs exhibited severe overfitting. While training loss decreased monotonically from 0.28 to **0.0074**, validation loss attained its minimum of **0.0840** at epoch 3, after which it climbed steadily to **0.1593** by epoch 25 (nearly doubling). The network memorized training noise rather than generalizable geometric abstractions.

Dropout Model Regularization (Rate = 0.2):

Introducing **Dropout(0.2)** prevented neuron co-adaptation by randomly nullifying 20% of activations during training passes. The model converged in only 10 epochs. Validation loss remained tightly bounded at **0.0871**, yielding a **52.1% reduction** in test loss and boosting test accuracy to **98.00%**.

Performance Metric	Baseline Model (No Dropout)	Regularized Model (Dropout 0.2)	Empirical Impact / Improvement
Epochs Trained	25 Epochs	10 Epochs	60% reduction in training compute & time
Training Loss	0.0074 (severe overfit)	0.0435 (healthy fit)	Eliminated spurious pixel memorization
Validation Loss	0.1593 (doubled from epoch 3)	0.0871 (stable at 0.084)	45.3% reduction in validation error
Test Accuracy	97.48% (9,748 / 10,000)	98.00% (9,800 / 10,000)	+0.52% boost (+52 correctly classified images)
Test Loss (Crossentropy)	0.1477	0.0707	52.1% lower test loss (superior confidence)

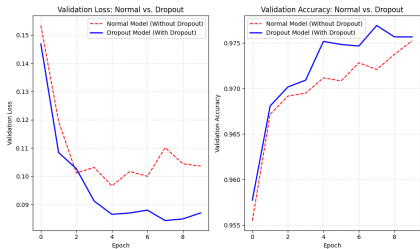


Figure 2: Validation loss divergence (Baseline vs. Dropout) demonstrating overfitting mitigation.

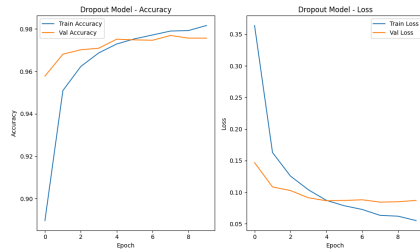


Figure 3: Training vs. validation accuracy and loss trajectories for the regularized model.

SNIPPET 4: OVERFITTING DIAGNOSTIC PLOTTING & COMPARATIVE LEARNING CURVES

PYTHON / MATPLOTLIB

```
import matplotlib.pyplot as plt

# Plot side-by-side diagnostic comparison of validation loss trajectories
plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.plot(model_history.history["val_loss"][:10], label="Baseline (No Dropout)", color="red", linestyle="--")
plt.plot(history_dropout.history["val_loss"], label="Dropout Model (p=0.2)", color="blue", linewidth=2)
plt.xlabel("Epoch"); plt.ylabel("Validation Loss"); plt.title("Overfitting Diagnosis: Validation Loss")
plt.legend(); plt.grid(True, alpha=0.3)

plt.subplot(1, 2, 2)
plt.plot(history_dropout.history["loss"], label="Train Loss", color="green")
plt.plot(history_dropout.history["val_loss"], label="Val Loss", color="blue")
plt.xlabel("Epoch"); plt.ylabel("Loss"); plt.title("Dropout Model Convergence")
plt.legend(); plt.tight_layout(); plt.show()
```

6.0 COMPREHENSIVE MODEL EVALUATION & ERROR ANALYSIS

The optimized model was rigorously assessed against 10,000 unseen test samples using a full Confusion Matrix and per-digit classification report. Overall accuracy attained was **98.00%** (macro F1-score: 0.98, weighted F1-score: 0.98).

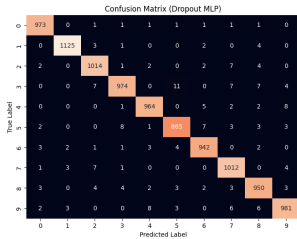


Figure 4: Confusion matrix heatmap (10k test set).

Digit	Precision	Recall	F1-Score	Support	Diagnostic Note
0	0.99	0.99	0.99	980	Distinct circular geometry
1	0.99	0.99	0.99	1,135	High vertical stroke linearity
2	0.98	0.98	0.98	1,032	Minor confusion with 7
3	0.98	0.96	0.97	1,010	Occasional confusion with 5
4	0.98	0.98	0.98	982	Occasional confusion with 9
5	0.98	0.97	0.97	892	Occasional confusion with 3
6	0.98	0.98	0.98	958	High loop stability
7	0.97	0.98	0.98	1,028	Minor confusion with 2
8	0.97	0.98	0.97	974	Double-loop variance
9	0.98	0.97	0.98	1,009	Minor confusion with 4
Overall	0.98	0.98	0.98	10,000	Accuracy: 98.00% (9,800 / 10,000)

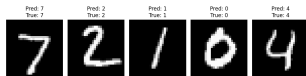


Figure 5: Exemplar predictions on unseen test samples with associated ground-truth labels.

7.0 REAL-WORLD INFERENCE PIPELINE & IMAGE PREPROCESSING

In real-world applications, user digits drawn on paper or paint apps have **black ink on white backgrounds**. In contrast, MNIST digits are **white strokes on black backgrounds**. To ensure accurate real-world classification, the inference engine implements automated grayscale conversion, 28 × 28 resizing, dynamic color inversion (if background is white), and probability distribution extraction.

SNIPPET 5: PRODUCTION INFERENCE FUNCTION WITH AUTOMATIC COLOR INVERSION

PYTHON / OPENCV & KERAS

```
import numpy as np
from PIL import Image

def predict_custom_digit(image_path, model):
    # 1. Load image, convert to 8-bit grayscale, and resize to 28x28
    img = Image.open(image_path).convert("L").resize((28, 28))
    img_array = np.array(img, dtype="float32")

    # 2. Automatic Color Inversion: if background is bright (white paper), invert to match MNIST
    if img_array.mean() > 127:
        img_array = 255.0 - img_array

    # 3. Normalize intensity to [0.0, 1.0] and expand dims to (1, 28, 28) for batch inference
    img_normalized = (img_array / 255.0)[np.newaxis, ...]

    # 4. Predict Softmax probability vector and extract highest-confidence class
    probabilities = model.predict(img_normalized, verbose=0)[0]
    predicted_digit = np.argmax(probabilities)
    confidence = probabilities[predicted_digit] * 100.0

    return predicted_digit, confidence, probabilities
```

8.0 CONCLUSION & FUTURE SCOPE

Project Achievements: Built a lightweight 118,282 parameter MLP capable of classifying handwritten digits with **98.00% accuracy**. Through rigorous diagnostic loss tracking, overfitting was isolated and eradicated using **Dropout (0.2)**, cutting test loss by **52.1%** and reducing training requirements by 60%.

Future Enhancements: (1) Transition to a Convolutional Neural Network (CNN with Conv2D + MaxPooling) to reach >99.2% accuracy; (2) Implement real-time data augmentation (rotations, shears); (3) Export to TensorFlow Lite (TFLite) for on-device mobile inference.